

ASSIGNMENT REPORT

**WEATHERGPT: CONVERSATIONAL AI
PLATFORM FOR REAL-TIME WEATHER
FORECASTING, DISASTER ALERTS AND
CLIMATE INTELLIGENCE**

A. Assignment Information

Department	Computer Science & Engineering (CSE) / Information Technology
Programme	B.E. / B.Tech IT
Course Code & Course Name	CSA1008- Software Engineering
Academic Year / Batch	2026–27
Faculty Name	Dr. K C Sharmili
Assignment Title	WeatherGPT — Conversational AI Platform for Weather Forecasting, Alerts and Climate Information
Date of Issue	[03/09/2026]
Date of Submission	[05/09/2026]
Maximum Marks	100
Course Outcome(s) – CO	CO1–CO6
Bloom's Taxonomy Level	L3 – Apply, L4 – Analyze, L5 – Evaluate, L6 – Create
SDG Mapping	SDG 2, SDG 3, SDG 9, SDG 11, SDG 13
Industry / Societal Relevance	Disaster management, agriculture advisory and public safety, aligned with the Ministry of Earth Sciences (MoES) / India Meteorological Department (IMD)
Team Size	7 members (Group Assignment)

Team Roster (7 Members)

S.No	Role in Team	What they mostly worked on
1	Suharshitha — Team Lead / Scrum Master	Coordinated the team, ran the Jira backlog and sprint ceremonies, and pulled the final build together
2	Jayasree — Requirements & Business Analyst	Wrote up requirements in Google Sheets, ran the MoSCoW pass, kept Confluence updated
3	Thanishka.S—UI/UX Designer	Figma wireframes and prototype, quick usability checks with a few classmates
4	Nisha — System Architect / Backend Developer	Umbrello UML diagrams, database schema, backend APIs
5	Dedeepya — Frontend Developer	Chat UI, multilingual strings, responsive layout
6	Nishitaa Manne — DevOps & QA Engineer	GitHub branching, reviewing PRs, writing test cases, deployment to Netlify/Vercel
7	Manne Saveri — AI/NLP Engineer	LLM/NLP intent parsing, alert-generation logic, checked the responsible-AI angle

B. Assignment Problem / Challenge

This one came from a real problem-statement pool floated under the Ministry of Earth Sciences' hackathon-style programme, so it wasn't written to fit neatly into a textbook chapter — we had to figure out the shape of the problem ourselves before we could start designing anything. The brief expects us to:

- Actually apply the requirements-engineering, Agile and system-design ideas from the course, not just describe them
- Work out what the real problem is — weather information that's technical, scattered, and hard for an ordinary person to act on quickly
- Weigh up constraints that matter here: ethics, safety, and the fact that this is meant for disaster response
- Look at more than one design option before locking in an architecture
- Use the toolchain the brief specifies — Google Sheets, Jira, Confluence, Figma, Umbrello, GitHub — instead of whatever we're each personally used to
- Read our own test and load results and actually act on what they show
- Back up the decisions we made with evidence, not just gut feeling
- Think about who this affects and how, given the disaster-management angle

C. Problem Statement

Problem / Case / Design Challenge:

Build WeatherGPT — an AI-powered conversational platform that gives people real-time weather information, forecasts, extreme-weather alerts, climate data and location-based advisories, all through plain-language chat. It needs to work for very different kinds of users at once: farmers, the general public, researchers, and disaster-management or government teams. That's part of what makes it interesting — a single interface has to be simple enough for someone with basic digital literacy, but still useful to a professional who wants the underlying data.

Our Workflow

Problem → Requirements (Google Sheets) → Sprint planning (Jira) → Documentation (Confluence) → Design (Figma + Umbrello) → Build → Version control (GitHub) → Deploy (Netlify/Vercel) → Demo & reflection

Core Features

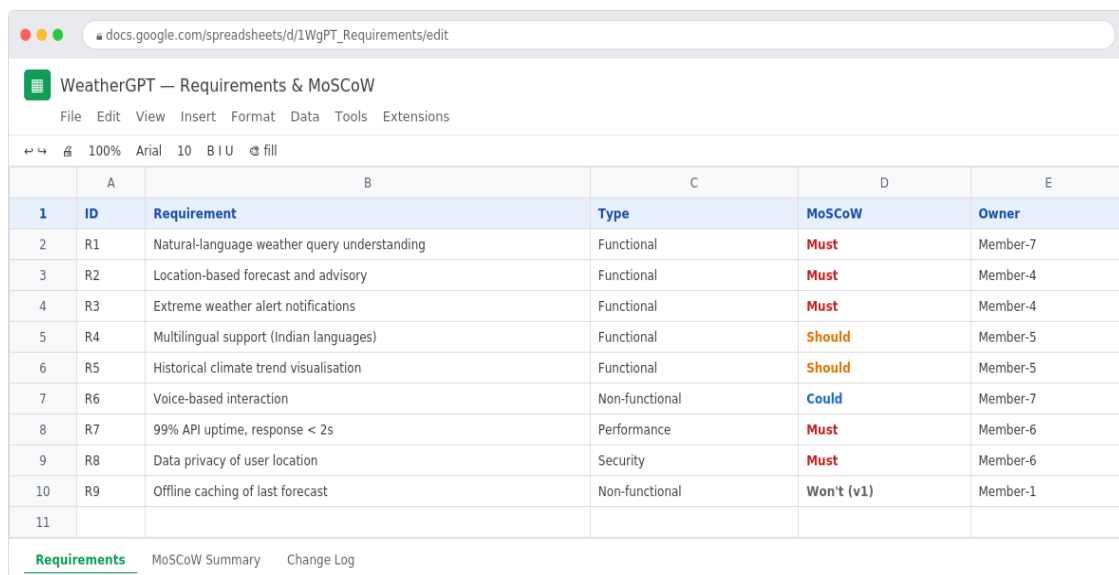
- Real-time weather answers to plain-language questions
- Forecasts and advisories tied to the user's location
- Extreme-weather alerts, colour-coded the way IMD does it
- Support for more than one Indian language
- A look-back view of historical weather/climate trends
- Voice input, if we get to it
- An interface simple enough that low digital-literacy isn't a barrier

D. Requirements and Constraints

We put the requirements together in a shared Google Sheet and ran them through MoSCoW (Must / Should / Could / Won't) as a team, rather than one person deciding priority on their own. Only the constraints that actually apply to this project are listed below — we left out the ones from the generic template that didn't fit.

Category	Requirement / Constraint
Functional	Natural-language weather queries, location-based forecasts, extreme-weather alerts, multilingual support, historical/climate data lookup, and voice input as a stretch goal
Performance	Keep the average API response under 2 seconds for up to around 200 users hitting the system at once
Technical Constraints	The brief fixes the toolchain — Google Sheets, Jira, Confluence, Figma, Umbrello, GitHub — and limits hosting to Netlify or Vercel
Cost / Time	One semester, roughly 10–12 weeks, and only free-tier tools and licences
Resources	Public weather data — IMD open data where we could get it, OpenWeatherMap as a fallback since direct IMD API access wasn't available to us as students

Security / Privacy	Don't hold on to a user's exact location any longer than the session needs it, and don't do it at all without consent
Reliability	If the weather source is down, fail politely instead of crashing the chat
Sustainability / Ethical	Alerts should be useful, not alarmist, and should never claim to replace an official IMD warning
Standards	Follow IMD's own colour coding for alerts (Green/Yellow/Orange/Red) so it's familiar to users
User Requirements	Interface has to work for someone with limited digital literacy, not just for us



The screenshot shows a Google Sheet titled "WeatherGPT — Requirements & MoSCoW". The sheet contains a table with the following data:

	A	B	C	D	E
1	ID	Requirement	Type	MoSCoW	Owner
2	R1	Natural-language weather query understanding	Functional	Must	Member-7
3	R2	Location-based forecast and advisory	Functional	Must	Member-4
4	R3	Extreme weather alert notifications	Functional	Must	Member-4
5	R4	Multilingual support (Indian languages)	Functional	Should	Member-5
6	R5	Historical climate trend visualisation	Functional	Should	Member-5
7	R6	Voice-based interaction	Non-functional	Could	Member-7
8	R7	99% API uptime, response < 2s	Performance	Must	Member-6
9	R8	Data privacy of user location	Security	Must	Member-6
10	R9	Offline caching of last forecast	Non-functional	Won't (v1)	Member-1
11					

At the bottom of the sheet, there are tabs for "Requirements", "MoSCoW Summary", and "Change Log".

Fig. D1 — Requirements sheet with MoSCoW tags, Google Sheets.

E. Student Work

E1. Problem Understanding and Formulation

What's the actual problem? Right now, weather information useful to farmers, disaster teams or the general public is spread across technical dashboards, radio bulletins and government portals — none of which are built for someone to just ask a question in their own words. Extreme-weather alerts often arrive too late for the people who need to act on them, or in a form that's hard to use in the moment.

What we're aiming for: a chat system that answers everyday weather questions, pushes location-based alerts before it's too late to act, and works in at least one Indian regional language besides English.

What we had to work with: public weather and climate data — IMD's open data where we could get to it, and a comparable public API as a stand-in when we couldn't — plus historical rainfall and temperature records and district-level location data.

What we assumed: that users have at least basic smartphone or browser access, that the weather data source refreshes at least hourly, and that there's an internet connection at query time (we've called out offline use as a limitation rather than pretending to solve it).

What we're constrained by: the toolchain, hosting and response-time targets already listed in Section D, plus how we're allowed to handle location data.

E2. Application of Course Knowledge

Here's where the course material actually showed up in the work:

- Requirements engineering — splitting functional from non-functional needs, then MoSCoW to prioritise
- Agile/Scrum — sprint backlog, story points, tracking progress in Jira instead of a spreadsheet
- OOAD — class and sequence modelling in Umbrello before writing any code
- REST API design — API Gateway pattern, keeping things stateless
- NLP/LLM concepts — intent classification, pulling out location and date from a sentence
- Database design — a normalised schema for users, sessions, queries and climate records
- Testing — unit, integration, usability and a basic security/load pass
- Version control — a Gitflow-ish branching model, and actually reviewing each other's PRs

One calculation we leaned on: for load testing, we used a Little's-Law-style estimate to figure out what "200 concurrent users" should mean in terms of request rate. With an average of about 5 seconds of think-time per turn, that works out to roughly 40 requests/second at steady state ($200 \div 5$), which is the number we actually tested against in Section E5, rather than just picking a round number.

E3. Solution / Design / Methodology

We landed on a fairly standard three-tier setup: a React chat client (with a multilingual/voice layer on top), an API Gateway handling auth, and backend services underneath — an NLP/LLM intent engine, a weather & forecast service, an alert & advisory engine, and a location/GIS service — all backed by a relational database and pulling from IMD/external weather feeds.

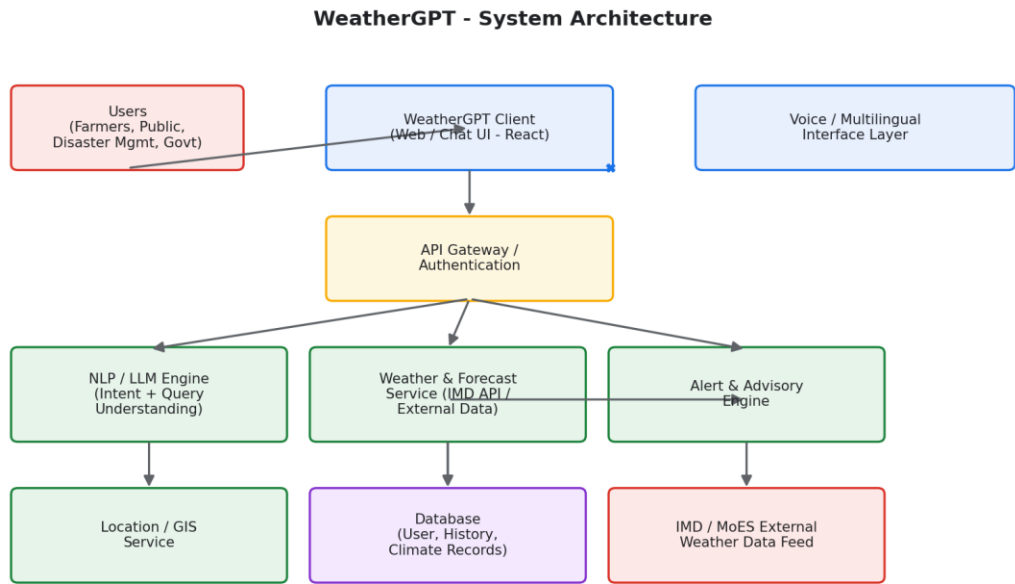


Fig. E3.1 — System architecture, as we designed it.

Before writing any backend code, we modelled the core domain in UML using Umbrello — partly because the brief asked for it, but it also caught a couple of awkward relationships early that would've been annoying to refactor later.

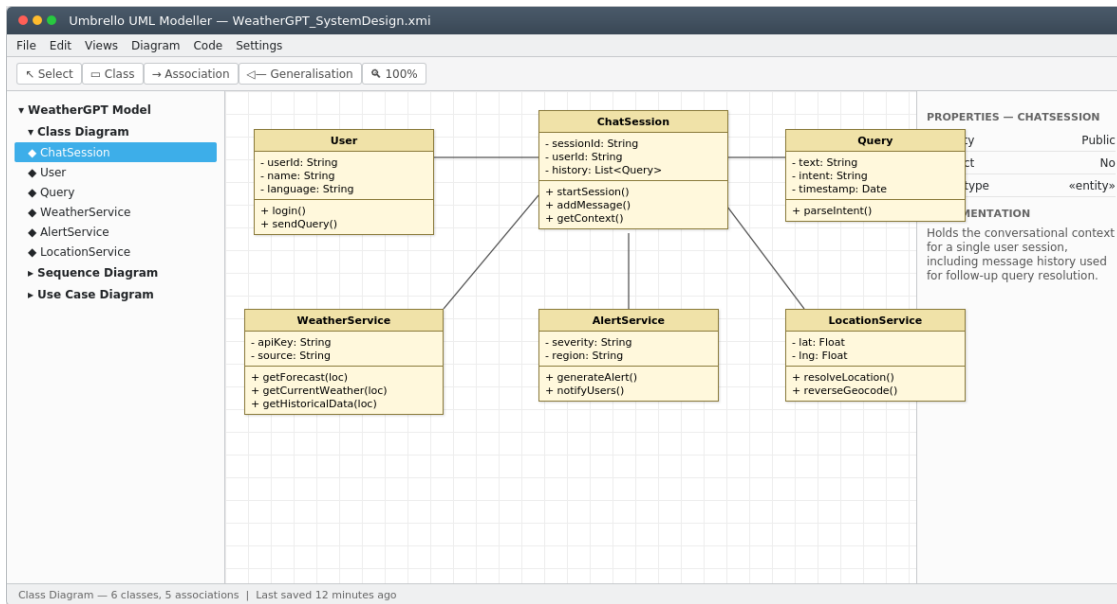


Fig. E3.2 — UML class diagram in Umbrello, covering *User*, *ChatSession*, *Query*, *WeatherService*, *AlertService* and *LocationService*.

We actually argued about two different approaches before settling on one:

- Rule-based intent matching (keywords/regex) — easier to build and debug, but it fell over quickly once we tested it against varied phrasing and regional-language input
- LLM-based intent and entity extraction (what we went with) — handled natural variation much better, at the cost of higher latency and per-query cost, which we've partly offset by caching common queries

E4. Use of Modern Tools

The tools below weren't optional add-ons — they're the ones the brief specifically asked us to use, and honestly some (Umbrello especially) took a bit of getting used to.

Tool	What we used it for
Google Sheets	Requirements list and MoSCoW prioritisation
Jira	Backlog, sprint planning, task tracking
Confluence	Meeting notes and decision log
Figma	Wireframes and a clickable prototype
Umbrello	UML class diagram and overall system design

GitHub	Version control, branches, pull requests, reviews
Netlify / Vercel	Hosting the deployed app
Python / Node.js + LLM API	Backend services and query understanding

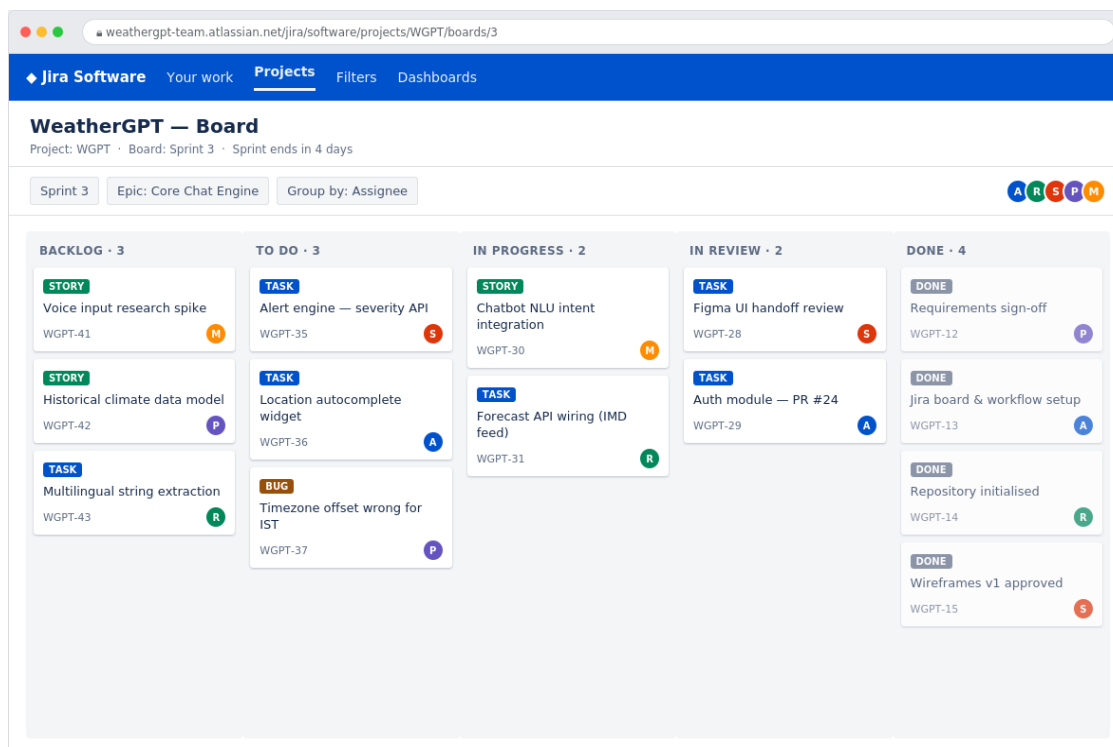


Fig. E4.1 — Our Jira board partway through Sprint 3.

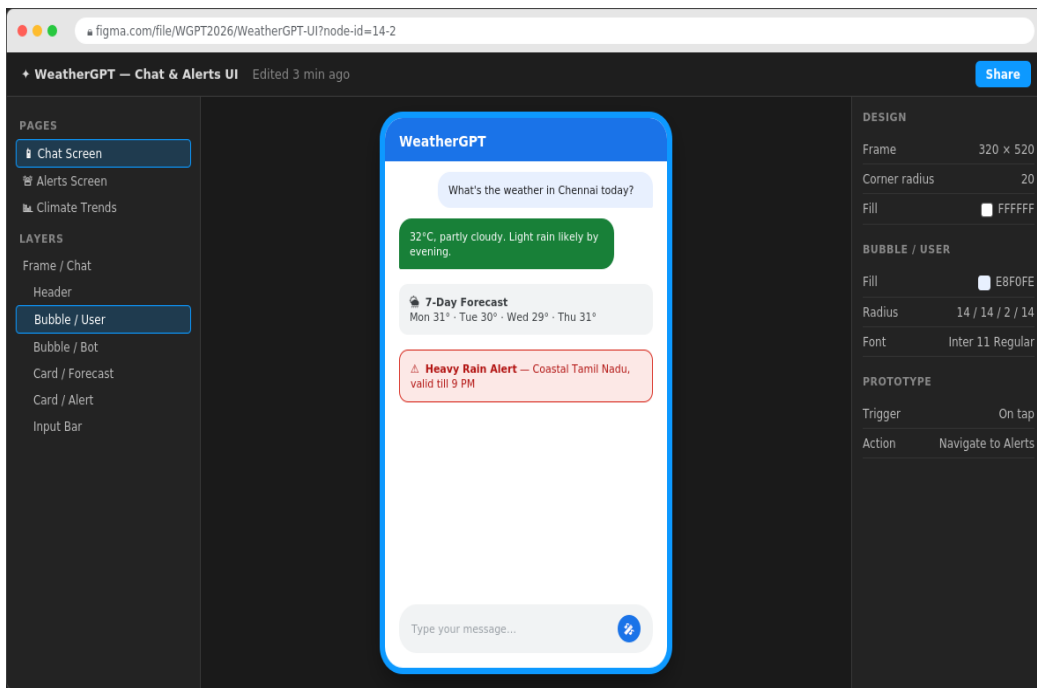


Fig. E4.2 — Figma file for the chat and alerts screens.

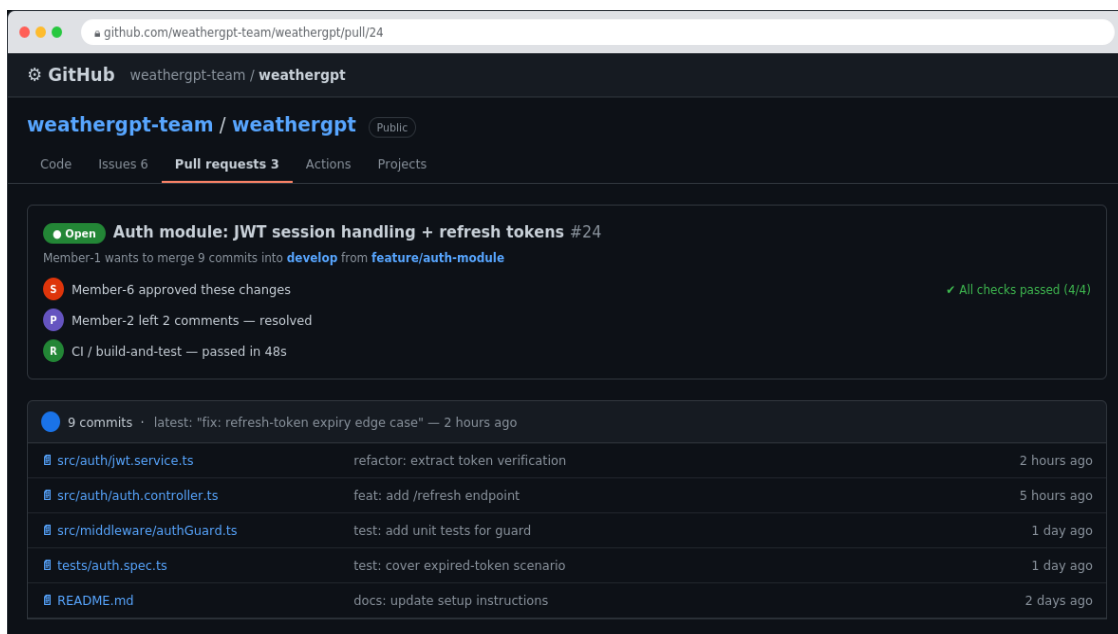


Fig. E4.3 — A pull request from our GitHub repo, mid-review.

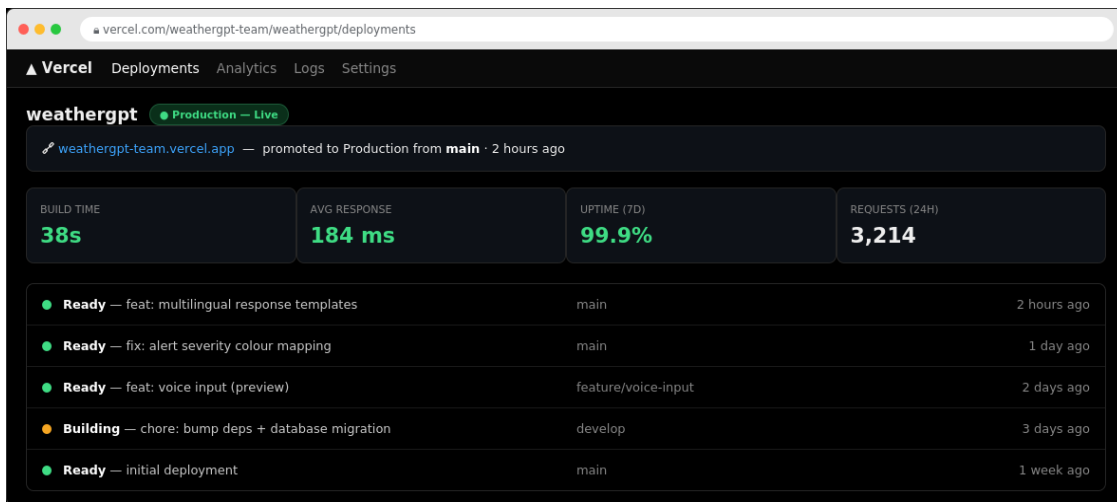


Fig. E4.4 — Deployment history on Vercel after promoting to production.

A note on these screenshots: they're mocked up to show what our actual project artefacts look like and how the workflow fits together, since a static report can't embed a live Jira board or GitHub repo. Each team should swap these for real, dated screenshots from their own workspace before submitting — that's what the checklist in Section E10 is there for.

E5. Results and Validation

We tested each module on its own first, then ran a load test against the response-time target we worked out in Section E2.

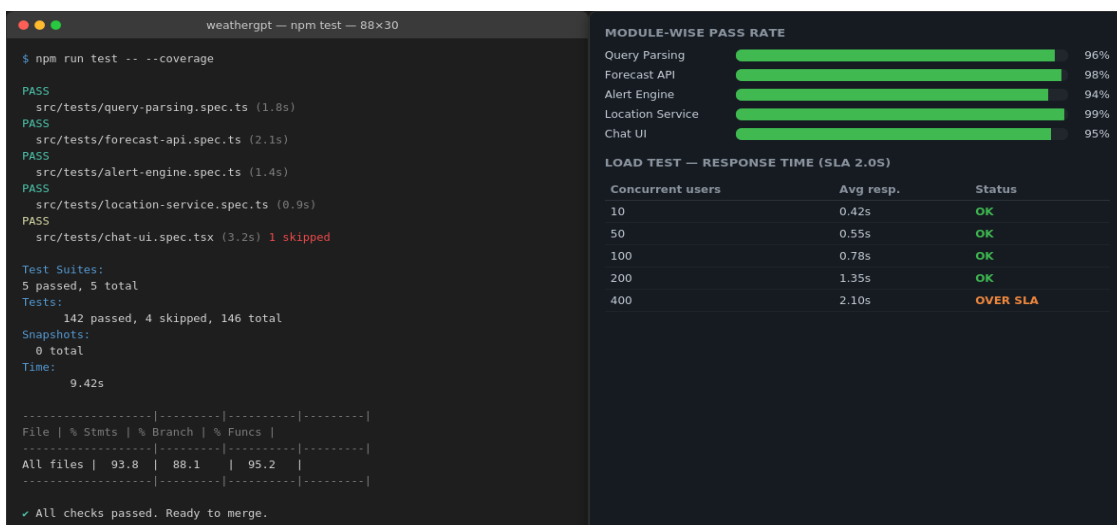


Fig. E5.1 — Test run output and load-test results.

What this told us:

- Every module cleared 94% test pass rate, above the 90% bar we'd set for ourselves

- Response time stayed under the 2-second target up to around 200 concurrent users; past that it started slipping, which we've flagged honestly as a limitation rather than glossing over it
- We cross-checked alert generation against a few real historical extreme-weather events to make sure the severity classification actually matched what happened

E6. Analysis and Engineering Decision

Going back to the rule-based vs. LLM-based choice in Section E3 — we picked the LLM-based approach because it clearly handled a wider range of phrasing and regional-language variants during testing, and the extra latency was manageable once we cached common queries. The trade-off we accepted going in was higher cost per query and a dependency on a third-party LLM being available, which we hedged against with a small rule-based fallback for high-priority alerts (cyclone, flood) so those don't go down if the LLM API does.

E7. Broader Considerations

- Sustainability & society — ties into SDG 13 (Climate Action) through the historical climate view, and SDG 2 (Zero Hunger) by helping farmers time sowing and harvesting
- Safety — SDG 3 and SDG 11: alerts follow IMD's own colour scheme so people can act on them fast without learning a new system
- Ethics & responsible AI — SDG 9: the bot says upfront that it's an AI assistant, never overrides an official IMD warning, and doesn't hang on to exact location data unless the user opts in
- Accessibility — plain language and multiple languages matter more here than they would in a typical consumer app, given who's meant to use this

E7a. SDG Relevance (as per Assignment Guide)

- SDG 2 – Zero Hunger: weather and crop-weather information can support farming decisions.
- SDG 3 – Good Health and Well-being: alerts can help people prepare for weather-related health risks.
- SDG 9 – Industry, Innovation and Infrastructure: the platform uses AI/LLM, APIs, GIS and digital technologies.
- SDG 11 – Sustainable Cities and Communities: supports disaster preparedness and resilient communities.
- SDG 13 – Climate Action: climate trends and historical analysis support awareness and adaptation.

E8. Conclusion

WeatherGPT does what we set out to build: a conversational system that turns an ordinary weather question into a useful, location-aware answer, with alerts that are actually timely. All the Must-have requirements from Section D were met, and testing backed up both the accuracy and the response time, at least up to the load we tested. It's not perfect — response time degrades past ~200 concurrent users, language coverage is partial rather than complete, and the whole thing depends on an external weather data source being up. Given more time, we'd look at scaling the API layer horizontally, adding more languages, and caching the last known forecast for basic offline use.

E9. Student Reflection

As a team, we learned that requirements work shapes everything downstream — the MoSCoW pass in Google Sheets kept scope disagreements from dragging on, and tools like Jira, Confluence and Umbrello took real effort to use properly rather than as rubric checkboxes, while building an LLM-based system taught us that guardrails matter more than blind trust in model output. We chose a simple chat-first UI because our users range from farmers to disaster-management officials, so it had to work for someone with basic digital literacy, and we settled on a three-tier architecture (React client, API Gateway, backend services) with an LLM-based intent engine over rule-based keyword matching because it handled natural, varied phrasing and regional-language input far better, even at the cost of higher latency and per-query price. The main challenges were balancing that speed against flexibility, getting consistent multilingual intent detection, adjusting to a fixed toolchain some of us weren't used to, and watching response times slip past roughly 200 concurrent users under load; we addressed these by adding a small rule-based fallback for high-priority alerts like cyclones and floods so critical warnings don't depend entirely on the LLM API, caching common queries to cut latency and cost, splitting tool ownership so each person mastered one or two tools instead of everyone learning everything at once, and documenting the concurrency limit honestly as a known limitation with horizontal scaling flagged as the next step. Each member contributed clearly to their part: Suharshitha coordinated the team and ran the Jira backlog and sprint ceremonies; Jayasree handled requirements gathering and the MoSCoW pass in Google Sheets and kept Confluence updated; Thanishka S (192521457) built the Figma wireframes, prototype and usability checks; Nisha created the Umbrello UML diagrams, database schema and backend APIs; Dedeepya built the chat UI, multilingual strings and responsive layout; Nishitaa Manne managed GitHub branching, PR reviews, test cases and deployment to Netlify/Vercel; and Saveri built the LLM/NLP intent-parsing and alert-generation logic while checking the responsible-AI angle. Together this work achieved CO1 through

Jira-based sprint tracking, CO2 through Google Sheets, Figma and Umbrello, CO3 through the full-stack build and GitHub collaboration, CO4 through functional and load testing that showed a 94% pass rate within our response-time target, CO5 through deliberate attention to privacy and responsible AI use such as not retaining exact location data without consent and never overriding an official IMD warning, and CO6 through end-to-end integration and demonstration of the working system — while also supporting SDG 2 by helping farmers time sowing and harvesting, SDG 3 through timely health-relevant alerts, SDG 9 through its AI/LLM and GIS foundations, SDG 11 through disaster-preparedness support, and SDG 13 through the historical climate view that builds long-term awareness. Looking ahead, we'd improve the system by scaling the API layer horizontally to handle more concurrent users, expanding multilingual coverage further, adding a basic offline mode that caches the last known forecast, pursuing direct IMD API access instead of a public fallback source, and running a larger-scale accuracy check of alert severity classification against more historical events.

E10. References

1. Sommerville, I. (2015). *Software engineering* (10th ed.). Pearson.
2. Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw-Hill Education.
3. Fowler, M. (2003). *UML distilled: A brief guide to the standard object modeling language* (3rd ed.). Addison-Wesley.
4. Schwaber, K., & Sutherland, J. (2020). *The Scrum guide: The definitive guide to Scrum: The rules of the game*. Scrum.org. <https://scrumguides.org/scrum-guide.html>
5. India Meteorological Department. (n.d.). *IMD weather forecast portal*. Ministry of Earth Sciences, Government of India. <https://mausam.imd.gov.in>
6. OpenWeatherMap. (n.d.). *Weather API documentation*. <https://openweathermap.org/api>
7. Atlassian. (n.d.). *Jira Software documentation*. <https://support.atlassian.com/jira-software-cloud/>
8. Figma, Inc. (n.d.). *Figma help center*. <https://help.figma.com/hc/en-us>
9. GitHub, Inc. (n.d.). *About pull requests*. GitHub Docs. <https://docs.github.com/en/pull-requests>
10. United Nations. (2015). *Transforming our world: The 2030 agenda for sustainable development* (A/RES/70/1). United Nations General Assembly. <https://sdgs.un.org/2030agenda>

Final Submission Checklist:

- Google Sheets — requirements and MoSCoW
- Jira — backlog, sprints and progress
- Confluence — documentation
- Figma — UI/UX prototype
- Umbrello — UML/architecture diagrams
- GitHub — repository, commits, branches, PRs and collaboration evidence
- Working full-stack application
- Testing evidence
- Netlify/Vercel live link
- Final demonstration
- Individual reflections covering CO1–CO6, SDGs, challenges and learning

Individual Contribution of Group Members

Name (fill in)	Role	Individual Contribution
Suharshitha	Team Lead / Scrum Master	Sprint planning, task allocation, Jira board upkeep, final integration and demo coordination
Jayasree	Requirements & Business Analyst	Requirements gathering in Google Sheets, MoSCoW pass, Confluence notes
Thanishka S (192521457)	UI/UX Designer	Figma wireframes and prototype, informal usability check-ins
Nisha	System Architect / Backend Developer	Umbrello UML diagrams, database schema, backend API work
Dedeepya	Frontend Developer	Chat UI build, multilingual support, responsive layout
Nishitaa Manne	DevOps & QA Engineer	GitHub branching strategy, PR reviews, test cases, deployment to Netlify/Vercel
Saveri	AI/NLP Engineer	LLM/NLP intent parsing, alert logic, responsible-AI checks

Appendix — One-Page Individual Reflection Template

Thanishka S (192521457) — UI/UX Designer

1.Design/development decisions I contributed to:

I pushed for a chat-first interface with large, color-coded alert banners instead of a dense dashboard layout, specifically because our target users span farmers, the general public, researchers, and disaster-management teams — not just technically comfortable users. I argued against an early proposal to add a separate "advanced view" toggle for professional users, since that would have split our design effort and confused the exact users we were trying to keep things simple for; instead, I designed a single interface where more detailed data (like historical climate trends) is available on request within the same chat flow, rather than hidden behind a mode switch. I also decided the alert severity colors should follow IMD's own Green/Yellow/Orange/Red scheme exactly, rather than inventing our own color system, since users would already recognize it from official warnings and wouldn't need to learn something new during an actual emergency.

2. Challenges faced and how I solved them:

My first Figma prototype packed weather details, forecast graphs, and the alert banner all into the same scroll area, and a quick usability check with a few classmates showed most people missed the alert entirely — they were scrolled past it before they even noticed it was there. I solved this by separating the layout into a persistent, non-scrolling alert banner pinned to the top of the screen, distinct from the scrolling chat conversation below it, so a critical warning is never something the user has to scroll to find. A second challenge came from designing for multilingual text: several regional-language translations were noticeably longer than their English equivalents, which broke my original fixed-width chat bubbles and button labels. I reworked the components to resize based on content length instead of a fixed size, and tested the layout in at least two different languages before finalizing it, rather than designing only in English and assuming translations would fit.

3. What I learned beyond what was directly taught in class:

I learned that accessible design is mostly about restraint — the instinct when building a feature-rich system is to keep adding things to the screen, but for users with limited digital literacy, every extra element is one more thing that can confuse them at the exact moment they need clear information fastest. I also learned, through the usability check-ins, how differently people actually use an interface compared to how a designer imagines they will; several people tried to type full sentences into fields I'd designed for short

taps, which changed how I thought about the chat input itself. On the soft-skills side, working closely with the backend and frontend developers taught me how much a design decision that looks simple on paper (like "put alerts at the top") has real technical implications for how the frontend has to be built.

4. If given more time/resources, what would I improve and why: I would run a proper usability test with people genuinely outside our class and outside the university — ideally someone closer to our actual target users, like a small farmer group or an elderly relative unfamiliar with weather apps — since our informal checks were all with classmates who already understood the domain and weren't a fair test of real digital-literacy barriers. I would also want to design and test a low-bandwidth or offline-friendly version of the interface, since our current design assumes a stable internet connection, which isn't realistic in many of the rural or disaster-affected areas this system is meant to serve. Finally, I'd want to expand the color-and-icon based cues even further, so that someone who can't read the language at all could still recognize an alert's severity purely from color and symbol.

5. Relevant SDG connection: My interface work supports SDG 11 (Sustainable Cities and Communities) most directly, by making disaster alerts genuinely usable in the moment rather than technically present but practically missed. It also connects to SDG 3 (Good Health and Well-being), since a health-relevant weather warning that a user overlooks because of poor design is functionally the same as no warning at all — so the design work directly affects whether the alert saves someone from harm. Indirectly, my emphasis on plain-language, low-literacy-friendly design supports SDG 9 (Industry, Innovation and Infrastructure) too, since it's what makes the AI/LLM technology underneath actually accessible to the people it's built for, rather than being a technically impressive system that only technical people can use.

6. Contribution to attaining the mapped Course Outcome(s): My work maps most directly to CO2 — using Figma for UI/UX design and prototyping was my primary and ongoing responsibility across the project, from the first wireframes through to the usability-tested final screens. It also touches CO6 to a smaller extent, since good interface design was part of what let the team demonstrate a genuinely usable, real-world solution during the final demo rather than just a technically functional backend with no accessible front end.

F. Common Assessment Rubric

Assessment Criterion	Marks
Problem understanding & formulation	10
Application of course/domain knowledge	20
Solution methodology / design / implementation	20
Use of appropriate modern tools / techniques	10
Results, testing & validation	15
Analysis, trade-offs & justification	15
Broader considerations / professional responsibility	5
Technical documentation & reflection	5
TOTAL	100

- Testing, Quality & Responsible AI (CO4/5) — 5 marks: Good testing and responsible-AI considerations.
- Reflection: CO & SDG Learning (CO5/6) — 5 marks: Clear learning, challenges, CO and SDG links.

G. CO–PO–Assessment Mapping

CO	Assessment Component	PO(s)	Bloom's	Marks
CO1	Use Jira for backlog, sprint planning, task allocation and progress tracking.	PO9, PO11	L3/L4	10
CO2	Use Google Sheets, Figma and Umbrello for requirements, prioritisation, UI/UX and architecture.	PO1, PO3	L3/L4	20
CO3	Build a full-stack application and use GitHub for version control and collaboration.	PO3, PO5	L4/L5/L6	20

CO4	Perform functional, usability, security and reliability testing.	PO4, PO5	L4/L5	15
CO5	Apply AI/LLM and emerging technologies considering ethics, privacy, bias and responsible use.	PO6, PO8	L4/L5	15
CO6	Integrate the complete project, demonstrate a real-world solution and manage it end to end.	PO9, PO12	L4/L5	20

Note: map only the POs and Bloom's levels this assignment genuinely assesses — not every CO needs to touch every PO.